LEX.UZ RAG API · BAJARILGAN ISH

Topshiriq bo‘yicha nima qilindi

Spetsifikatsiyaning har bir bandi, qanday bajarilgani, va yo‘lda topilgan xatolar.

ENDPOINT	§7 CHECKLIST	TEST	HIT@5	FAITHFULNESS
5 / 5	14 / 14	76	0.978	1.000

TO‘QILGAN MANBA

0

1. Topshiriq nima edi

RAG API — test spetsifikatsiyasi (lex.uz) hujjati bitta batch endpoint talab qiladi: 10 tagacha savol qabul qilib, har biri uchun javob, lex.uz manbalari (hujjat + modda) va topilgan kontekst bo‘laklarini qaytarishi kerak. Maqsad — RAG ning ikki qismini **alohida** baholash: qidiruv to‘g‘ri moddani topdimi, va javob shu moddaga sodiqmi.

BOSHLANG‘ICH HOLAT

Loyihada bitta endpoint bor edi: `POST /api/chat` — bitta savol qabul qilib `{answer, sources}` qaytaradi. Spetsifikatsiyadagi `/v1` endpointlarning **birortasi ham yo‘q edi**. Qidiruv oddiy vektor qidiruvi edi, sitata tekshiruvi yo‘q, baholash mexanizmi yo‘q, test yo‘q.

2. Qurilgan narsalar

BESHTA ENDPOINT

ENDPOINT	VAZIFASI	SPEC
`POST /v1/rag/batch`	1–10 savol, javob + manba + kontekst	MAJBURIY
`GET /v1/rag/batch/{id}`	Natijani qayta olish, async poll	MAJBURIY
`GET /v1/meta`	Model, embedding, indeks versiyasi, snapshot	MAJBURIY
`GET /v1/health`	Servis ishlayaptimi	MAJBURIY

`POST /v1/rag/query`	Bitta savol, debug uchun	ixtiyoriy

Barcha §5 status kodlari: `400` validatsiya (11 savol, takroriy `id`, bo‘sh savol), `401` / `403` auth, `409` bir xil `batch_id` boshqa mazmun bilan, `429 + Retry-After`, `202` async + `GET` bilan olish. Savol darajasidagi kodlar: `RETRIEVAL_FAILED`, `LLM_TIMEOUT`, `LLM_ERROR`, `CONTEXT_TOO_LONG`, `INVALID_LANGUAGE`.

GIBRID QIDIRUV VA HAQIQIY RERANKING

Ikki qidiruv birga ishlaydi, chunki har biri boshqasi uddalamagan joyda ishlaydi. Dense (e5 vektor) ma’noni topadi: *“ishdan bo‘shatishsa nima qilish kerak” → mehnat shartnomasini bekor qilish* moddalari. BM25 esa kam uchraydigan aniq atamani topadi: *o‘g‘irlik* → sarlavhasi *O‘g‘rilik* bo‘lgan modda.

Chunking **modda bo‘yicha**, belgi soni bo‘yicha emas — shu sababli `citations[].article` aniq, taxmin emas. Indeksda **7 383 modda**: Konstitutsiya + 20 kodeks + 4 qonun.

SITATA TEKSHIRUVI — ASOSIY MEXANIZM

Model matn emas, strukturalangan JSON qaytaradi: javob, va har bir da’vo uchun qaysi kontekstdan olgani va **harfma-harf ko‘chirgan** parcha. “So‘raldi” degani “kafolatlandi” degani emas, shuning uchun har bir sitata chiqishda qayta tekshiriladi:

```
quote kontekst matnining aynan bir bo‘lagimi?   yo‘q → sitata tashlanadi
ko‘rsatilgan kontekst haqiqatan qaytarilganmi?  yo‘q → sitata tashlanadi
chunk_id KONTEKSTDAN olinadi, modeldan emas
hech qaysi sitata qolmasa → javob no_answer ga tushiriladi
```

Shuning uchun to‘qilgan manba “tavsiya etilmaydi” emas — **strukturaviy imkonsiz**. Model ko‘rmagan narsaga havola berishning yo‘li yo‘q.

PROVAYDERGA BOG‘LIQ BO‘LMASLIK

Faqat uchta kod yo‘li bor: ikki native API (Anthropic, Gemini) va qolgan hammasi gaplashadigan OpenAI protokoli. Shuning uchun DeepSeek, OpenRouter, Together, Mistral, xAI,

Fireworks, Cerebras, Ollama, LM Studio — jadvaldagi bitta qator, kod emas. `RAG_LLM_BACKEND=auto` bo‘lsa, qaysi kalit mavjud bo‘lsa shu provayder tanlanadi, ya’ni `.env` ga istalgan bitta kalit tashlash yetarli.

BAHOLASH MEXANIZMI

- **100 savollik to‘plam** — spec §6 tarkibi bo‘yicha aniq: faktik 35, raqamlar 15, ko‘p modda 15, amaldagi tahrir 10, kirill/rus 10, javobsiz 10, noaniq 5. Har batch ga har kategoriyadan aralashtirilgan.
- **Runner** — spec §1 ssenariysini so‘zma-so‘z bajaradi: 10 batch ketma-ket, retryable xatoda 5s/15s/45s bilan qayta urinish (bir xil `batch_id`, ya’ni server idempotentligi ishlaydi), `Retry-After` ni hurmat qilish, `202` async yo‘li, uzilsa qoldirgan joyidan davom etish.
- **Scorer** — §6 metrikalarini maqsadlarga qarab hisoblaydi, va o‘lchash mumkin bo‘lmaganini “o‘lchanmaydi” deb aytadi, taxmin qilmaydi.
- **76 test** — §7 checklist ning har bir bandi uchun. “API testga tayyormi?” savoliga kod o‘qib emas, testni yurgizib javob beriladi.

---

3. Yo‘lda topilgan xatolar

Bu bo‘lim eng muhim: quyidagilarning hammasi **kodni o‘qib emas, ishga tushirib** topilgan. Har biri real o‘lchov, real so‘rov yoki spetsifikatsiyani qayta o‘qib solishtirish natijasida chiqqan.

Oxirgi audit alohida: javob spec §4 ning majburiy maydon ro‘yxatlari va §5 ning status jadvaliga qarshi qatorma-qator tekshirildi — o‘z da’volarimga ishonmasdan. Barcha majburiy maydonlar joyida, ikkala strukturaviy qoida ham bajarilgan. Ikki narsa esa bajarilmagan edi, ular quyida.

> **RRF fusion to‘g‘ri moddani ko‘mib tashlagan**
>
> Boshida ikki qidiruvning *o‘rnini* qo‘shgandim (reciprocal rank fusion — standart usul). Bu korpusda noto‘g‘ri: yuridik moddalar lug‘atni baham ko‘radi, shuning uchun e5 cosine’lari zichlashadi — bitta savolning 40 nomzodi **0.79–0.81** oralig‘ida. Ya’ni dense *tartibi* deyarli tasodif. BM25 farqi esa keng va ma’noli: sarlavhaga aniq moslik **12.5**, keyingisi **7.9**.
>
> **Tuzatish:** RRF ball kattaligini tashlab yuboradi — aynan shu yerda farqlaydigan signalni. Absolyut ballga o‘tildi: `1.0·dense +`

`0.8·leksik + 0.3·kodeks`. *O‘g‘irlik* savoli 5-o‘rindan 1-o‘ringa chiqdi.

Kirill savollar leksik qidiruvda hech nima topmagan

Korpus lotin yozuvida. *ўғрилик* deb so‘ralsa, u javob beradigan modda bilan (*o‘g‘rilik*) bitta ham umumiy token baham ko‘rmaydi, ya’ni BM25 nol beradi va faqat dense qoladi — bu yetmaydi.

Tuzatish: kirill → lotin transliteratsiya tokenizatsiya ichida. Spec 10 ta kirill/rus savolini talab qiladi, ya’ni bu o‘lchanadigan talab edi.

Kodeks bonusi dense vaznini ikki baravar oshirgan

Savolda kodeks nomi aytilsa, unga doirali qidiruv ham ishlaydi. Dastlab bu natija dense ballga *qo‘shimcha* hissa qo‘shardi — ya’ni kodeks aniqlanganda dense vazni jimgina 1.0 dan 1.6 ga chiqardi. Natijada cosine 0.845 bo‘lgan umumiy modda (*Jinoyatning tushunchasi*) savolning hal qiluvchi so‘zi sarlavhasida turgan moddani bosib ketgan.

Tuzatish: bonus *tekis* qilindi (0.3) — bir kodeks ichidagi nomzodlar orasida o‘zaro bekor bo‘ladi va qidiruvlarning o‘zi hal qiladi.

Indeksda 60 ta buzuq bo‘lak, degenerat sitata bergan

Kolleksiyada eski skraper konveyerining bo‘laklari ham bor edi — butun hujjat belgi soni bo‘yicha kesilgan, modda strukturasi yo‘q. Ulardan qurilgan sitata `doc_id: "lex:"`, `chunk_id: "lex:#art-"` degan bo‘sh qiymat berardi, va spec majburiy qilgan `article` / `part` maydonlarini to‘ldira olmaydi.

Tuzatish: `/v1` faqat modda darajasidagi korpusni xizmat qiladi, chiqarilgan son esa `/v1/meta` da ko‘rinadi — yashirilmaydi. Kunlik ish endi alohida kolleksiyaga yozadi, ya’ni qayta ifloslantira olmaydi.

QQS savoli noto‘g‘ri «javob yo‘q» bergan

Real model bilan sinaganda chiqdi. *“QQS stavkasi necha foiz?”* → `no_answer`. Tekshirsam: qidiruv to‘g‘ri moddani **1-o‘rinda** topgan (Soliq kodeksi 258-modda), model esa rad etgan — va *to‘g‘ri* qilgan. 258-moddada “soliq stavkasi 12 foiz” deyilgan, “QQS” so‘zi **yo‘q**. U QQS stavkasi ekani faqat *QO‘SHILGAN QIYMAT SOLIG‘I* sarlavhasi ostida turganidan bilinadi. Shu kodeksda 13 ta modda “Soliq stavkalari” deb nomlangan va hammasi stavkani shunday nomsiz aytadi.

Sabab: korpus bo‘limni `"X BO‘LIM."` deb saqlagan — yuqoridagi parser keyingi qatordagi *nomni* tashlab ketgan. Ya’ni moddaga ma’no beradigan sarlavha kontekstda yo‘q edi. **Tuzatish:** xom matndan 24 ta bo‘lim nomi tiklandi, kontekstga `location` qatori qo‘shildi, `qqs` statut so‘zlariga bog‘landi. Natija: *“QQS stavkasi 12 foiz [1]”* harfma-harf sitata bilan — va Marsdagi savol hali ham `no_answer`.

Provayder kvotasi tizimning xatosi deb hisoblangan

Bepul tarif daqiqada 8 000 token beradi, bitta savol esa ~2 500 token oladi. Ya’ni `429` — oddiy hol. Lekin u doimiy `LLM_ERROR` bo‘lib qolardi, ya’ni bepul kalitdagi tizim pullik kalitdagi *aynan shu* tizimdan ancha past baho olardi.

Tuzatish: retryable xato vaqt budjeti ichida qayta urinadi, provayder `Retry-After` bersa shuni hurmat qiladi. Budjet 45s — Groq ~33s so‘raydi, dastlabki 25s esa undan sal kam bo‘lib, o‘tib ketadigan limitni baholangan xatoga aylantirgan edi.

Scorer qisman run’da maxrajni butun to‘plam deb olgan

20 savolga javob kelganda ham 90 ta javobi bor savolga bo‘lgan, ya’ni Hit@5 noto‘g‘ri **0.19** chiqqan. Bu “tizim yomon” degan xulosaga olib kelardi, aslida esa “run to‘liq emas” edi.

Tuzatish: metrikalar faqat javob kelgan savollar ustidan hisoblanadi, qamrov esa alohida ko‘rsatiladi — qisman run endi `PARTIAL RUN` deb boshlanadi.

Airflow papkasi haqiqiy kutubxonani to‘sib qo‘ygan

Papka `airflow/` deb nomlangani uchun `import airflow` bizning bo‘sh papkamizga tushardi — `version` yo‘q

`__file__` = None. Ya’ni DAG ning o‘z importlari **hech qachon ishlashi mumkin emas edi**, va `apache-airflow` hech qaysi requirements faylida ham yo‘q edi.

Tuzatish: `airflow/ → orchestration/` , va `apache-airflow requirements-scraper.txt` ga qo‘shildi.

Spec’ni qayta o‘qishda: `INDEX_UNAVAILABLE` hech qachon chiqmagan

§5 bu kodni `RETRIEVAL_FAILED` bilan yonma-yon sanaydi, va ikkisi borligining sababi aynan farq: biri “vektor baza bilan aloqa yo‘q”, ikkinchisi “shu savolning qidiruvi yiqildi” deydi. Kodda esa har qanday qidiruv xatosi — Qdrant butunlay o‘chiq bo‘lgan holat ham — `RETRIEVAL_FAILED` deb qaytarilardi.

Nega muhim: tester servis o‘chiqligini bilmay, savolning o‘zida xato izlab vaqt sarflaydi. **Tuzatish:** ulanish va timeout xatolari alohida tasniflanadi.

Spec’ni qayta o‘qishda: butun batch yiqilsa HTTP 200 qaytgan

Ishchi oqim har qanday istisnoni ushlab, `status: "failed"` yozuvini saqlardi va endpoint uni *natija kabi* qaytarardi. §5 esa servis xatosini `500` / `503` / `504` ga bog‘lagan — va aynan shu kod testerga 5s, 15s, 45s bilan qayta urinishni aytadi.

Nega muhim: 200 bilan `run_eval.py` nol javobli batch’ni natija deb saqlab, keyingisiga o‘tardi — ya’ni o‘tib ketadigan nosozlik run’da jimgina teshik qoldirardi. **Tuzatish:** endi `503` qaytadi; `GET` esa saqlangan `failed` yozuvini berishda davom etadi, §5 aynan shunga ruxsat beradi.

Yana to‘rtta kichik, lekin real xato

Provayder default’lari eskirgan — `gemini-2.0-flash` ham `llama-3.3-70b-versatile` ham sinalgan kalitlarda mavjud emas edi; `list_models.py` yozildi, u kalit nimaga kira olishini ko‘rsatadi.
Groq ikki xil JSON xatosi — faqat “Failed to validate JSON” ushlanardi, “Failed to generate JSON” esa umumiy 400 ga tushib, matndagi “token” so‘zi sababli `CONTEXT_TOO_LONG` va *qayta urinilmaydigan* deb belgilanardi. Real batch’da bitta savol shundan yo‘qolgan.
Avtomatik aniqlash Ollama’ni tanlagan — localhost preset’i kalit talab qilmaydi, shuning uchun hech narsa sozlanmaganda ham “topilgan” bo‘lardi va har savol ulanish xatosi bilan yiqilardi.
Testlar real modelga murojaat qilgan — `verify.py .env` dagi provayderni pytest ichki protsessiga o‘tkazib yuborgan: 274 sekund, har yurgizishda pul, va bepul tarif limiti “test yiqildi” bo‘lib ko‘rinadi. Endi kontrakt testlari offline’da, generatsiya esa alohida qadamda tekshiriladi — 24 sekund.

---

4. O‘lchangan natija

100 savol, real HTTP endpoint orqali, 10 batch ketma-ket:

METRIKA	NATIJA	MAQSAD	HOLAT
Hit@5 (hujjat)	0.978	≥ 0.90	O‘TDI
Context precision	0.960	≥ 0.60	O‘TDI
Faithfulness	1.000	≥ 0.95	O‘TDI
Citation accuracy	0.966	≥ 0.90	O‘TDI
To‘qilgan manba	0	0	O‘TDI
Eskirgan norma	0	0	O‘TDI

Latency p50 / p95	0.23s / 0.27s	≤ 5s / ≤ 12s	O‘TDI
Xato ulushi	0.000	≤ 0.02	O‘TDI
Hit@5 (modda), MRR	–	≥ 0.80 / 0.70	GOLD KERAK
Answer correctness	–	o‘rtacha ≥ 1.6	GOLD KERAK

Latency generatsiyani hisobga olmaydi — provayder ulanganda p50 1–3s oralig‘ida bo‘ladi, maqsad esa 5s, ya’ni zaxira katta.

Uch tilda qo‘lda tekshirilgan 5 savol: **5/5 birinchi o‘rinda** (lotin, kirill, rus).

---

5. Qilinmagan narsalar va nega

HIT@5 (MODDA), MRR, ANSWER CORRECTNESS — O‘LCHANMAGAN

Test to‘plamida `expected_article` va `expected_answer` ustunlari **ataylab bo‘sh**. Savol qaysi moddaga tushishi “kerak” ekanini yozish uchun statutlarni o‘qish kerak; agar men ularni tizimning *o‘z chiqishidan* to‘ldirsam, baholash tizimni o‘zi bilan o‘zini tekshirgan bo‘lardi — bu bema’nilik. Spec ham test to‘plamini tester tomonida deb belgilagan. Shu ikki ustun to‘ldirilsa, scorer uchala metrikani avtomat hisoblaydi, kod o‘zgarmaydi.

DOC_STATUS DOIM «AMALDA»

Indeksdagi 25 hujjatning hammasi kuchda, ya’ni “eskirgan norma” metrikasi **yiqila olmaydi**. U yerdagi `0` — tekshiruv ishlayotganining dalili emas. Buni haqiqatan sinash uchun indeksga kuchini yo‘qotgan hujjat kerak.

EDITION_DATE HUJJAT BO‘YICHA, MODDA BO‘YICHA EMAS

Korpus — amaldagi konsolidatsiyalangan matnning sanali snapshot’i, tahrir tarixi yo‘q. Shuning uchun yagona asosli tahrir belgisi — snapshot oyi. Modda darajasidagi o‘zgartirish sanalari uchun lex.uz revizion skrape kerak.

TEMPERATURE VA SEED

Provayderga qarab farq qiladi: Groq ikkisini ham qabul qiladi

(ya’ni spec talabini so‘zma-so‘z bajaradi), Gemini faqat `temperature`, Anthropic SDK 2.x esa sampling parametrlarini butunlay qabul qilmaydi. Ikkisi ham har doim javobda qaytariladi va javob keshi kalitiga kiradi, ya’ni bir xil so‘rov bir xil javob beradi. Qidiruv — spec aynan takrorlanuvchanlik uchun tekshiradigan qism — konstruksiya bo‘yicha deterministik. `GET /v1/meta` qaysi holat amal qilayotganini aytadi.

---

6. Raqamlar

NIMA	HAJM
Yangi yozilgan kod (backend + scripts + tests)	~5 250 qator
Hujjat (README, TESTING, provenance)	~1 170 qator
Test holatlari	76
Indekslangan modda	7 383
Hujjat (kodeks + qonun + Konstitutsiya)	25
Test to‘plami	100 savol / 10 batch
Qo‘llab-quvvatlanadigan provayder	11 + custom
Commit	9
O‘chirilgan o‘lik kod va balast	~700 qator + 15 MB

TOZALASH

Eski chat UI va uning konveyeri (`frontend/`, `backend/rag.py`) o‘chirildi: u *aynan shu* savollarga boshqa qidiruv kodi orqali javob berardi, ya’ni bitta savol qaysi eshikdan kirganiga qarab ikki xil javob qaytarishi mumkin edi — demo qilish chalkash, baholash esa chalg‘ituvchi. Shuningdek korpus bilan kelgan 15 MB uchinchi tomon balasti (ko‘rib chiqilmagan skill fayllari, upstream’ning demo kodi, xom matn) va ishlatilmaydigan test artefaktlari ketdi.

---

7. Qanday tekshiriladi

```
docker compose up -d                      # Qdrant
python ingest/ingest_corpus.py            # bir marta: indeks
uvicorn backend.main:app --port 8000      # server

python scripts/verify.py --quick          # «tayyormi?» — bitta buyruq
python scripts/run_eval.py --run-id r1    # 100 savol, 10 batch
python scripts/score_eval.py --run-id r1  # §6 metrikalari
```

`verify.py` ketma-ket tekshiradi: muhit, Qdrant, modda indeksi, uch tilda qo‘lda tasdiqlangan 5 savol, §7 checklist (76 test), va sozlangan provayder — keyin bitta xulosa chiqaradi. `FAIL` bo‘lsa, tuzatish yo‘lini ham aytadi.

Batafsil yo‘riqnoma `TESTING.md` da: Linux, macOS va Windows uchun alohida o‘rnatish, har bir endpoint uchun nusxa-ko‘chir buyruqlar (PowerShell variantlari bilan), 15 ta xato holati jadvali, va *“bular xato emas, shunday bo‘lishi kerak”* bo‘limi.

Oxirgi holat: `verify.py --quick` → 6 o‘tdi, 0 yiqildi. 76 test o‘tadi. Server ishlayapti, Groq provayderi bilan real javob beradi. Ikki ochiq shart: baholash uchun pullik kalit (bepul tarif 100 savol uchun qattiq) va test to‘plamidagi ikki gold ustun.